

AhamX Bodhi

Where I Becomes Infinite

Layer Normalisation

Module 3.2: Transformer Architecture Deep-Dive
Prof. Sashikumaar Ganesan | IISc Bengaluru



Session Overview

What You Will Learn

- Why normalisation is needed in deep networks
- The LayerNorm formula step by step
- Post-LN vs. Pre-LN and why Pre-LN won
- RMSNorm: the modern simplification
- Role in training stability for large LLMs

Concept at a Glance

- **Duration:** 8 minutes
- **Bloom's Level:** B3 – Apply
- **Difficulty:** L3 – Advanced Intermediate
- **Module:** 3.2 Transformer Deep-Dive
- **Prerequisite:** Decoder-Only Architecture



Learning Objectives

By the End of This Session You Will Be Able To

1. **Explain** the problem that normalisation solves in deep Transformer training
2. **Write** the LayerNorm formula and identify the role of each term
3. **Distinguish** Batch Norm, LayerNorm, and RMSNorm
4. **Compare** Post-LN and Pre-LN placement and explain why Pre-LN is preferred
5. **Identify** LayerNorm and RMSNorm in GPT-4, Llama 3, and Mistral architectures



Prerequisites and Connections

You Should Already Know

- Residual connections (skip connections)
- Vanishing and exploding gradient problems
- Batch Normalisation concept (helpful not required)
- Forward pass of a Transformer block

This Concept Unlocks

- Feed-Forward Networks in Transformers
- Full Transformer block assembly
- Training stability for large pre-training runs
- QLoRA: layer-wise quantisation awareness





The Problem: Internal Covariate Shift

Why Normalisation Is Needed

- As a neural network trains, the distribution of activations at each layer shifts after every weight update — called **internal covariate shift**
- Later layers must constantly adapt to a shifting input distribution — slowing learning and requiring lower learning rates
- For deep networks (32–96 layers in LLMs), this effect compounds: activations can grow or shrink exponentially across layers
- Without normalisation, gradients either **vanish** (activations collapse to zero) or **explode** (activations grow uncontrollably)

Normalisation stabilises the distribution of activations, enabling faster training with higher learning rates and more stable gradient flow.



Batch Normalisation vs. Layer Normalisation

Batch Normalisation (2015)

- Normalises across the **batch dimension**
- Statistics (mean, var) computed per feature, across all examples in a batch
- Requires a minimum batch size for stable statistics
- Fails for variable-length sequences
- Cannot be used with batch size 1 (inference)

Layer Normalisation (2016)

- Normalises across the **feature dimension**
- Statistics computed per token, across all features of that token
- Works with batch size 1; ideal for sequence models
- No dependency on other examples in the batch
- Natural fit for Transformers and RNNs



The Layer Normalisation Formula

Two Steps: Normalise, Then Re-Scale

For a token representation $\mathbf{x} \in \mathbb{R}^d$:

$$\mu = \frac{1}{d} \sum_{i=1}^d x_i, \quad \sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2$$

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \varepsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$

$\varepsilon \approx 10^{-5}$ prevents division by zero. γ (scale) and β (shift) are **learned parameters** per feature, initialised to 1 and 0 respectively. They allow the network to undo normalisation when optimal — the network learns how much normalisation to apply.



Post-LN vs. Pre-LN Placement

Post-LN (Original Paper 2017)

- LayerNorm applied *after* residual:
 $\text{LN}(x + F(x))$
- Gradients from the loss flow through LayerNorm before reaching attention
- Requires warm-up learning rate schedules to avoid divergence
- Used by: original Transformer, early BERT models

Pre-LN (Modern Standard)

- LayerNorm applied *before* attention:
 $x + F(\text{LN}(x))$
- Gradient from the loss passes through the residual path unobstructed
- More stable training; no warm-up required; higher LR feasible
- Used by: GPT-2, GPT-3, GPT-4, Llama, Mistral, Claude



RMSNorm: Simplified Layer Normalisation

Root Mean Square Normalisation (Zhang & Sennrich 2019)

- **Hypothesis:** The re-centring (mean subtraction) in LayerNorm is unnecessary; only the re-scaling matters for training stability
- Removes the mean computation entirely; normalises by the RMS value only

$$\text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2}, \quad y_i = \frac{x_i}{\text{RMS}(\mathbf{x})} \cdot \gamma_i$$

No μ , no β — 7% fewer operations per normalisation call. Empirically matches or exceeds LayerNorm performance. **Used in:** Llama 2/3, Mistral, Falcon, Gemma, PaLM 2.



Worked Example: LayerNorm on a Single Token

Tracing LayerNorm for $\mathbf{x} = (2, 4, -2, 0)$, $d = 4$

Step 4: Scale and shift ($\gamma = [1, 1, 1, 1]$, $\beta = [0, 0, 0, 0]$): Output = $\hat{x}$ (identity transform when parameters are initialised)

Result: The token representation now has zero mean and unit variance. As training proceeds, γ and β are updated to learn the optimal scaling.





GenAI Application: Enabling Billion-Scale LLM Training

Layer Normalisation as a Training Infrastructure Component

- **GPT-3 (175B parameters):** Pre-LN placement was the key change that allowed training 96 attention layers stably without gradient explosion
- **Llama 3.1 (405B):** RMSNorm at every sublayer; no warm-up schedule required due to Pre-LN gradient stability
- **Google PaLM (540B):** Uses Pre-LN with RMSNorm; trained on 6144 TPU v4 chips — normalisation stability is critical when thousands of devices sync
- **Checkpoint recovery:** Because Pre-LN provides stable gradient norms, LLM training jobs can resume from checkpoints without gradient divergence

Without stable normalisation, training runs at this scale would diverge routinely.



GenAI Application: Fine-Tuning with LoRA

Normalisation Layers During Efficient Fine-Tuning

- In LoRA fine-tuning (Low-Rank Adaptation), only the attention weight matrices receive low-rank updates; **LayerNorm/RMSNorm parameters are always fully tuned**
- The normalisation parameters (γ, β in LN; γ in RMSNorm) are small (2 vectors of size d_{model}) but have high impact on output distribution
- **QLoRA**: Quantises all other layers to 4-bit but keeps normalisation parameters in full 16-bit precision for training stability
- **Instruction tuning**: Updating normalisation parameters during RLHF allows the model to shift its output register from base-model to chat style



GenAI Application: Inference Optimisation

Fusing Normalisation for Production Inference Speed

- **Operator fusion:** In TensorRT, DeepSpeed, and vLLM, RMSNorm is fused with the subsequent linear projection into a single GPU kernel — eliminating a memory round-trip and reducing latency by 10–20%
- **FP8 quantisation (H100 GPUs):** Normalisation keeps activations in a numerically stable range, enabling safe quantisation to 8-bit floats without per-tensor calibration steps
- **Speculative decoding:** The draft model and target model share the same normalisation type (RMSNorm); consistent scaling ensures the draft distribution closely matches the target, increasing acceptance rate
- **PagedAttention (vLLM):** Stable activation norms from RMSNorm allow batching many requests without per-request numerical stability concerns



GenAI Application: Normalisation in Multimodal Models

LayerNorm Across Text, Image, and Audio Modalities

- **CLIP (OpenAI):** Both the text encoder and vision encoder use LayerNorm; normalising both modalities to a shared scale enables cosine similarity comparisons
- **Stable Diffusion:** The UNet and the text encoder use separate LayerNorm instances; Group Norm in UNet ResNet blocks handles variable spatial resolutions
- **Whisper:** LayerNorm in both the audio encoder and text decoder; normalising the spectrogram feature space helps with multilingual robustness
- **Vision Transformers (ViT):** Patch embeddings are LayerNorm'd before each attention block; the same Pre-LN placement as text Transformers



Common Misconceptions

Clearing Up Confusion About Layer Normalisation

- **“LayerNorm is the same as BatchNorm”**: BatchNorm normalises across the batch dimension (requires large batches); LayerNorm normalises across the feature dimension (works with batch size 1).
- **“RMSNorm is just a faster approximation that sacrifices quality”**: Empirical results (LLaMA, PaLM) show RMSNorm matches or slightly exceeds LayerNorm at the same parameter count — the mean subtraction was simply unnecessary.
- **“LayerNorm prevents all training instability”**: LayerNorm addresses covariate shift but not all instabilities. Loss spikes can still occur from learning rate warmup, gradient clipping thresholds, or data quality issues.



LayerNorm vs. RMSNorm: Side-by-Side

Property	LayerNorm	RMSNorm
Mean subtraction	Yes	No
Variance computation	Yes	No (uses RMS)
Learned parameters	γ, β	γ only
Operations per call	More	~7% fewer
Performance	Strong	Equal or better
Memory	Slightly higher	Slightly lower
Used in	BERT, GPT-2, T5	Llama 2/3, Mistral, PaLM 2

Summary



Layer Normalisation — Key Takeaways

1. Layer Normalisation normalises activations across the feature dimension for each token independently — no batch size dependency
2. The formula normalises to zero mean and unit variance, then applies learned scale (γ) and shift (β) parameters
3. Pre-LN placement (before attention) provides superior gradient stability over Post-LN and is now the standard in all major production LLMs
4. RMSNorm omits mean subtraction, reducing computation by $\sim 7\%$ with equal or better empirical performance — now preferred in Llama, Mistral
5. Stable normalisation is a prerequisite for training models at 10B–400B+ parameters and is a key component in fine-tuning strategies like LoRA



What This Concept Unlocks

Next in Module 3.2

- Feed-Forward Networks in Transformers
- Full Transformer Implementation (coding)

Later in the Course

- LoRA and QLoRA Fine-tuning
- Training LLMs from Scratch
- Inference Optimisation (vLLM, TensorRT)
- Quantisation and FP8 Training

Thank You

Layer Normalisation | Module 3.2 | AhamX Bodhi